

软件过程与管理第一次课后作业

姓名：柏威雄 学号：2023141461072

一、所选项目概述

项目名称：校园图书借阅管理系统（Web 管理端+微信小程序端）

项目属性：高校软件工程课程实践项目

团队规模：2 人学生开发团队（1 人负责后端+数据库，1 人负责前端+小程序+测试）

开发周期：总时长 2 周

核心目标：完成图书信息管理、读者借阅/归还、逾期计费、数据统计等核心功能，满足课程实践验收要求

项目特点：需求固定无频繁变更、无商业交付压力、无长期运维需求、小团队短周期、轻量化交付

二、自定义项目软件过程

本过程以软件生命周期全链路为框架，结合小型项目特点细化为 7 个核心过程，明确每个阶段的目标、活动、输入、输出与执行要求：

1. 项目启动与规划过程

阶段目标：明确项目边界、分工、进度与风险，搭建项目执行基础

核心活动：

1. 界定项目功能范围与非功能要求
 2. 拆分任务并完成团队分工
 3. 制定 2 周开发进度表
 4. 识别技术、时间、协作类风险并制定简易应对方案
- 输入：课程实践任务书、教师指导要求
 - 输出：《项目计划书》《团队分工表》《项目风险清单》
 - 执行要点：扁平化分工，无专职项目经理，两人共同决策

2. 需求工程过程

阶段目标：完整、清晰、无歧义地定义系统需求，避免开发偏差

核心活动：

1. 收集图书管理业务需求与课程验收需求
 2. 拆解功能需求（图书管理、借阅管理、用户管理、统计分析）
 3. 梳理非功能需求（易用性、本地部署、数据准确性）
 4. 绘制用例图，完成需求确认
- 输入：项目计划书、业务场景说明
 - 输出：《需求规格说明书》《系统用例图》
 - 执行要点：需求一次性确认，无后期大范围变更

3. 软件设计过程

阶段目标：完成系统架构、模块、数据库与接口设计

核心活动：

1. 概要设计：确定 B/S 架构、前后端分离模式
 2. 模块设计：划分后端接口模块、前端页面模块、小程序核心模块
 3. 数据库设计：设计读者表、图书表、借阅记录表、逾期记录表
 4. 接口设计：定义前后端数据交互接口规范
- 输入：需求规格说明书
 - 输出：《系统概要设计文档》《模块详细设计》《数据库 ER 图+表结构》《接口文档》
 - 执行要点：设计轻量化，以草图+简易文档为主，不追求复杂格式

4. 软件实现过程

阶段目标：完成编码开发，实现全部设计功能

核心活动：

1. 搭建开发环境（后端框架、数据库、小程序开发工具）
 2. 按模块编码实现，同步完成单元自测
 3. 每日代码同步与简易版本管理
 4. 修复自测发现的基础问题
- 输入：设计文档、开发规范

- 输出：项目源代码、可运行程序包、自测记录
- 执行要点：前后端并行开发，每日沟通进度，快速解决协作问题

5. 软件测试过程

阶段目标：验证功能符合需求，修复致命/严重缺陷

核心活动：

1. 编写测试用例（覆盖核心功能、边界场景）
 2. 执行单元测试（模块级）、集成测试（系统级）、功能测试
 3. 记录 Bug，分级修复并回归验证
 4. 确认无影响演示的缺陷
- 输入：源代码、需求规格、测试用例
 - 输出：《测试用例》《测试报告》《Bug 修复记录》
 - 执行要点：以手动黑盒测试为主，聚焦核心功能可用性

6. 部署与交付过程

阶段目标：完成系统部署，准备课程验收演示

核心活动：

1. 本地服务器环境部署
 2. 系统联调与最终验证
 3. 制作演示 PPT，梳理演示流程
 4. 完成课程项目提交
- 输入：测试通过的系统版本、文档资料
 - 输出：部署包、演示 PPT、项目交付包
 - 执行要点：本地部署即可，无需上线公网，满足演示要求

7. 项目收尾过程

阶段目标：整理资料、复盘总结，完成项目归档

核心活动：

1. 汇总所有过程文档、代码、设计图
2. 复盘开发中的问题与改进点

- 3. 完成项目归档打包
- 输入：项目全周期产出物
- 输出：《项目总结报告》《项目归档包》
- 执行要点：精简总结，聚焦实践收获与过程优化

三、软件过程定义来源

本次过程定义以 ISO/IEC 三大标准为理论依据，分别提取核心框架、评估思想、改进原则，形成适配项目的自定义过程：

1. 核心框架来源：ISO/IEC 12207

该标准是软件生命周期过程的基础标准，将软件过程分为基本过程、支持过程、组织过程三大类。

本次直接引用其基本过程中的开发过程，选取规划、需求、设计、实现、测试、部署、收尾全链路，搭建本次项目的过程主体框架。

引用其支持过程中的文档编制、质量保证，为每个阶段设定文档产出与质量检查要求。

作为本次过程定义的核心骨架，保证流程符合国际标准的生命周期逻辑。

2. 过程可落地性来源：ISO/IEC 15504（SPICE）

该标准是软件过程评估与改进模型，核心是定义过程能力、强调过程可测量、可验证、可追溯。

参考其过程产出物可检查的思想，为每个阶段明确输入与输出，避免流程空转。

借鉴其过程分级评估思路，用简易方式检查阶段完成度，确保每个过程落地执行。

解决标准流程“落地难”的问题，让过程适配学生项目的实际执行。

3. 过程适配性来源：ISO/IEC 33001

该标准是软件过程改进框架，核心强调以业务目标为导向、过程适用性剪裁、持续优化。

遵循其“过程适配项目场景”的核心原则，不僵化套用标准，而是结合项目特点调整。

采用其轻量化、目标导向的改进思路，以“完成课程实践、功能可用”为目标设计流程。

为过程剪裁提供理论依据，保证剪裁后的流程合规且实用。

四、过程剪裁思路

本次剪裁严格遵循“保留核心、删减冗余、适配场景”原则，结合项目小团队、短周期、课程实践三大特点，对标准过程进行针对性调整：

1. 剪裁核心原则

1. 目标导向：以课程实践验收为核心，不追求商用项目的全流程标准化。
2. 轻量化：减少文档工作量、简化评审流程，降低小团队执行成本。
3. 完整性：不砍掉核心开发链路，保证软件过程的逻辑完整。
4. 可执行：流程贴合 2 人团队协作模式，无冗余环节。

2. 具体剪裁内容

1. 删减组织级管理过程

去掉商用项目必备的合同评审、供应商管理、第三方质量审计。
取消正式配置管理库、变更控制委员会（CCB），仅用 Git 做简易版本控制。
需求无重大变更，省略复杂的变更申请、审批流程。

2. 删减全生命周期运维过程

省略软件运行、长期维护、退役、用户培训、售后支持*等流程。
项目仅需演示交付，无后期运维需求，直接裁剪该类过程。

3. 简化文档与评审过程

删减标准格式的冗余文档，用简易文档、草图、表格替代正式报告。
合并需求评审、设计评审，2 人直接沟通确认，无需多轮会议评审。

4. 合并低价值阶段

合并“项目规划”与“启动”环节，压缩前期准备时间。
合并“部署准备”与“交付演示”，减少阶段衔接耗时。

3. 保留的核心过程

1. 完整开发链路：规划→需求→设计→实现→测试→交付→收尾，保证开发逻辑闭环。
2. 核心质量活动：需求确认、单元测试、集成测试，确保功能无致命缺陷。
3. 基础追溯能力：保留核心文档、代码版本记录，保证过程可查、可复盘。

4. 适配性调整

1. 团队适配：2人扁平化协作，无层级分工，省略专职管理角色。
2. 周期适配：压缩每个阶段时长，阶段间无缝衔接，适配2周短周期。
3. 目标适配：聚焦功能实现与演示，省略性能优化、安全加固等非核心要求。